

Contents

Abstract	4
Keywords	4
Streszczenie	5
Słowa kluczowe.....	5
1. Introduction	6
1.2. Compilers, interpreters, and performance	8
2. Aim and scope	11
2.1. Aim of the project	11
2.2. Scope	11
2.3. Specific requirements	12
3. Computer Chess	15
3.1. General issue	15
3.2. Game Representation	16
3.3. Board, Pieces, Moves, Squares, and Colors.....	19
3.4. Move Generation.....	19
4. Implementation.....	22
4.1. Tools.....	22
4.1.1. Bitboard Editor	22
4.1.2. Pyrs, a Python to Rust transpiler	23
4.1.3. Windows Performance Analyzer, Windows Performance Recorder	24
4.2. Design patterns	25
4.2.1. Generators	26
4.2.2 Inheritance	27

4.3. PGN representation	27
4.4. Optimizations	30
4.4.1. Build Configuration.....	30
4.4.2 Reading a file	31
4.4.3. Selecting the hash function	32
4.4.4. Optimizing individual snippets	32
4.5. Syzygy tablebases	33
4.6. UCI communication	35
5. Use cases	36
5.1 GUI chess game	36
5.2 Chess game statistics	37
5.3 Chess engine backend	37
6. Results	39
6.1. Python using the regular interpreter	39
6.2. Python, using PyPy	40
6.3. Rust Chess	42
6.4. Rust Chess, multi-threaded.....	43
6.5. Comparison.....	44
6.6. Game speed analysis.....	45
6.7. Other chess analysis software.....	46
7. Conclusions	49
List of tables	50
List of figures	50
Bibliography.....	52

|

Abstract

Variants of chess have been played for hundreds of years now. It is a game of vast complexity and ever since the rise of digital computing, humans create computer programs to play and analyze chess games. In the last decades, computer chess managed to surpass the abilities of chess masters and now computers are an essential tool for game preparation, finding tricky sequences of moves and self-improvement, used by grandmasters as well as chess enthusiasts.

This thesis aimed to develop a chess analysis library with a focus on performance fixated on the PGN files parsing speed. To achieve that a Python library called python-chess was chosen as a foundation and reference.

Although complete in functionality, the python-chess implementation is very slow mainly due to having been written in interpreted as opposed to a compiled language. For that reason, Rust, a modern, compiled programming language was chosen for this project. Its syntax and functionality allow for easy translation of Python code and its unique memory management, verbose error messages, and integrated package manager open doors to fast development.

The final product, later called Rust Chess, implements all the core features of the original library. Its features include move generation, handling common chess move, position, and game representation formats, detecting various events such as checks or stalemates, and communication with chess engines via UCI protocol.

This thesis exhibits the struggles of optimizing software for speed, explains how to represent a concept of a game of chess using only measures provided by a computer, and benchmarks Rust Chess' speed as compared to python-chess.

Despite preserving the layout and methods used in python-chess, performance increases were very significant leading to Rust implementation being two orders of magnitude faster than its original counterpart. For the 489 MB file, Rust Chess is 160 times quicker than python-chess using Python interpreter. Long compilation times and losing the possibility of using the library in the interactive mode are the inevitable drawbacks of the chosen approach. This makes the Rust library not a better alternative to python-chess, but a viable tool that fills a gap left by its predecessor.

Keywords

chess, game parsing library, game analysis, PGN format, Rust

Streszczenie

Różne wersje szachów są znane ludzkości od setek lat. Szachy są grą o ogromnej złożoności, więc od czasów rewolucji związanej z rozpowszechnieniem elektroniki cyfrowej tworzone są programy komputerowe do gry w szachy i analizy partii. Takie programy w ostatnich latach osiągnęły poziom przewyższający umiejętności mistrzów szachowych, a dzisiaj komputery są niezbędnym narzędziem w przygotowaniach do gier, znajdowaniu zaskakujących linii czy kontynuacji otwarć oraz do poprawy własnych umiejętności. Takie programy używane są zarówno przez arcymistrzów szachowych, jak i amatorów.

Celem tej pracy dyplomowej jest stworzenie wydajnej biblioteki do analizy partii szachowych, ze szczególnym naciskiem na szybkość parsowania plików PGN. Biblioteka python-chess napisana w języku Python posłużyła za źródło odwołań i szkielet dla tworzonej biblioteki.

Python-chess jest biblioteką rozbudowaną i dojrzałą, jednak ze względu na język, w którym została napisana, jej szybkość pozostawia wiele do życzenia. Z tego powodu Rust, nowoczesny język kompilowany został wybrany do implementacji tego projektu.

Produkt końcowy, w dalszych częściach tekstu nazywany Rust Chess, zawiera wszystkie kluczowe moduły zawarte w oryginalnej bibliotece. Jego możliwości to m.in. generowanie ruchów, obsługa popularnych formatów ruchów, pozycji i reprezentacji gier, wykrywanie zdarzeń typu szach, pat lub mat czy komunikacja z silnikami szachowymi przez protokół UCI.

Niniejsza praca przedstawia rozwiązywanie trudności związanych z optymalizacją oprogramowania pod kątem szybkości, tłumaczy, jak przedstawić pojęcie szachów korzystając wyłącznie ze środków, którymi dysponuje komputer i porównuje szybkość biblioteki Rust Chess z biblioteką python-chess.

Pomimo zachowania struktury i układu kodu takiego jak w python-chess, różnica wydajności jest znacząca. Implementacja w języku Rust jest 160 razy szybsza dla pliku o rozmiarze 489 MB. Nieuniknioną wadą tego rozwiązania jest utrata możliwości korzystania z biblioteki w trybie interaktywnym oraz długie czasy kompilacji. Podsumowując, Rust Chess nie jest lepszą alternatywą dla python-chess, lecz użytecznym narzędziem, które wypełnia lukę pozostawioną przez swój wzorzec.

Słowa kluczowe

szachy, parsowanie gier, analiza partii, format PGN, Rust

1. Introduction

Chess is a two-player game of perfect information [1], meaning that both players know the starting position, what pieces are on the board, available moves, etc. There is no secret information and no randomness involved in chess. There exists at least one optimal strategy, the sequence of moves providing the best possible result assuming the perfect play of the opponent. Therefore, either black or white is able to force a draw or a win from starting position. Finding the optimal strategy, however, is nearly impossible. The lower bound of the number of games of 40 moves¹ or 80 plies² is estimated to be 10^{120} by Claude Shannon (1950) [2] or 10^{123} by Victor Allis (1994) [3] and forced mates³ as long as 549 moves were found [4]. Since calculating the best progression of moves from the start until the end of the game is unobtainable, chess players analyze distinct positions and openings up to specified depth or consider simple positions with a limited number of pieces on the board to calculate the true value of the game. Being a tedious task, computer engineers managed to automate it, ultimately creating chess analysis software and chess engines.

¹ In game theory a move consists of two plies, or a turn taken by two players.

² A turn taken by only one player, a half-move

³ Disregarding the fifty-move rule



Figure 1. Chess analysis software (<https://lichess.org/analysis>)

Transferring chess analysis to the digital domain allows extracting data from a huge number of games. PGN⁴ is a file format used to store chess games in bulk and parsing PGN files fast is a dominant issue, described throughout this thesis.

⁴ Portable Game Notation, a human friendly format for storing a whole game of chess. It stores the moves in plain text described in Standard Algebraic Notation. (https://www.thechessdrum.net/PGN_Reference.txt)

```

[Event "London m2"]
[Site "London"]
[Date "1851.??..??"]
[Round "11"]
[White "Bird, Henry Edward"]
[Black "Horwitz, Bernhard"]
[Result "1/2-1/2"]
[WhiteElo ""]
[BlackElo ""]
[ECO "C66"]

1.e4 e5 2.Nf3 Nc6 3.Bb5 d6 4.c3 Bd7 5.d4 Nf6 6.O-O Be7 7.d5 Nb8 8.Bd3 O-O
9.Ne1 b6 10.f4 Bg4 11.Qc2 exf4 12.Bxf4 Nbd7 13.Nd2 Bh5 14.Nef3 h6 15.Rae1 Bg6
16.Nd4 Nc5 17.Bc4 a6 18.b4 Nb7 19.Bd3 Qd7 20.N3f3 Rfe8 21.c4 a5 22.Nh4 axb4
23.Nxg6 fxg6 24.Nc6 Nc5 25.e5 Nxd3 26.Qxd3 dxe5 27.Bxe5 Bc5+ 28.Kh1 Kh7 29.Bxf6 Rxe1
30.Rxe1 gxf6 31.d6 Qxd6 32.Qf3 Ra3 33.Qe4 f5 34.Qe8 Re3 35.Rxe3 Bxe3 36.Qxe3 Qxc6
37.Qe7+ Kg8 38.Qxb4 Qd6 39.Qb3 Kf7 40.a4 Qc5 41.h3 f4 42.Qd3 Qe3 43.Qd7+ Qe7
44.Qd5+ Qe6 45.Qd4 g5 46.c5 Ke7 47.Qg7+ Kd8 48.cxb6 Qxb6 49.Qf8+ Kd7 50.Qg7+ Kc8
51.h4 Qa5 52.Qxh6 Qe1+ 53.Kh2 Qg3+ 54.Kg1 1/2-1/2

```

Figure 2. Portable Game Notation, an exemplary game

1.1.

1.2.1.1. Compilers, interpreters, and performance

To explain the differences between compilers and interpreters, it might be best to first quote their definitions.

“Simply stated, a compiler is a program that can read a program in one language - the source language - and translate it into an equivalent program in another language - the target language”

“An interpreter is another common kind of language processor. Instead of producing a target program as a translation, an interpreter appears to directly execute the operations specified in the source program on inputs supplied by the user” [5]

Python is a high-level language ordinarily used in tandem with an interpreter, whereas Rust is a low-level compiled language. The distinction between a high-level and a low-level language and how to fit a certain language into one of these categories is wildly disputed [6]. However, the fact that Rust is considered a lower-level language and allows to write code at a lower level of abstraction than Python is undisputable. This fact will be clearly demonstrated in chapters comparing Rust Chess to python-chess running using compiled alternatives of Python.

Multiple factors are contributing to the difference in the time of execution of programs written in these languages. For once, Rust is statically typed, and in contrast, Python is dynamically typed. In statically typed language types of variables are known at the compile time. The type must be introduced by a programmer or inferred by the compiler. Dynamic type checking associates an object with a type at runtime. It does so every time an object is used. This comes at a cost because all the checks that would have been done at the compile time, now contribute to the execution time of a program. Python in its nature, is acutely dynamic, allowing for schemes such as evaluating a string as an executable code and returning its result, but flexibility like this causes optimization problems. Another aspect is memory management. Rust slyly forces the programmer to manually manage memory, and the predominant part of memory safety tests happens at compile time. Python uses an automatic form of memory management called garbage collection. Garbage collectors are very common, even in languages much faster than Python, but the fact is that they require computation to realize tasks, which can be done by a programmer without excess computing [7] [8] [9].

Compiled Languages utilize a compiler to translate a program from source language to the target language. Usually, the compiler translates the source language to very efficient assembly or machine code. Once done, the result can be used any number of times without the computation overhead related to the translation. Interpreted languages do not get translated to machine code before being run. Instead, an interpreter parses, translates and executes the instructions directly, while running. Parsing and translating during runtime significantly contribute to the completion time. The created overhead is the reason why compiled code is generally faster than interpretation [10]. This results in python-chess ready to be used right away, where Rust Chess needs to be compiled first. The compilation times, with compiler optimizations enabled, can exceed one minute, making the wait time certainly significant. However, it only takes PGN files reaching orders of megabytes to obliterate Python's head-start advantage.

2. Aim and scope

2.1. Aim of the project

The goal of the thesis project is to create a chess analysis library written in a fast, compiled programming language with capabilities similar to that of the python-chess, a popular Python chess library. The purpose of making so is to improve on performance. Such a library should enable users to create software on top of it, for the purpose of e.g. creating a complete GUI⁵ chess software or extracting data from large databases of games. This library is not a chess engine. Multiple chess engines such as stockfish, which strength is superior to that of any grandmaster are open-source, and developing a competitive chess engine nowadays is beyond the extent of a thesis paper for a single, time-constrained student. Instead, the library communicates with existing engines over the Universal Chess Interface.

The final objective of Rust implementation of the library is to provide a fully functional, alternative, that thrives in demanding applications, requiring raw performance, where flexibility, responsiveness, and on-demand startup speed are not the main concerns.

The python-chess is excellent in the use case, in which Python's potential to be used as a scripting language is harnessed and it would be difficult to match opportunities that it contributes. For this reason, Rust Chess is intended to implement the exact same framework, keeping the division and names of variables and methods the same. It enables programmers to switch between languages without hassle, saving them from learning characteristic methods once again. This approach will facilitate users to use these products side by side, using Rust implementation for power applications and Python library for quick tests and designing functionality. Such design allows utilizing both libraries' best aspects.

2.2. Scope

For the reasons introduced in the previous chapter, Rust Chess will not implement supplementary modules intended to mimic python-chess's functionality inherited from Python's scripting capabilities. It will incorporate only the core modules related to standard chess, which in fact constitute the vast majority of the original library's source code. It indicates that the other less common chess variants will not be supported.

⁵Graphical User Interface

Another somewhat omitted utility is querying the chess tablebases on local hardware. The library is designed to evaluate positions using external chess engines, which very often have access to tablebases themselves. For the same reason, the opening book queries are not in the scope of Rust Chess. Nonetheless, for user convenience, an abstraction compatible with the program's structures, used to make queries to online tablebase API will be implemented. This approach gives the impression of being a valuable midway between utilization and how much of an endeavor it is to carry out the project.

2.3. Specific requirements

List of characteristics to fulfill include:

- abstract structures for representing basic chess entities, namely the board, pieces, and moves,
- making moves and undoing moves,
- keeping track of the previously made moves,
- printing the current position in a terminal, in human-friendly form,
- understanding all the chess rules, including lesser-known but a part of the official FIDE⁶ laws of chess, such as the en-passant, fifty-move rule or the threefold and fivefold repetition,
- generating legal and pseudo-legal moves,
- creating a move from the UCI⁷ as well as SAN⁸ format,
- setting the board position using FEN⁹ format,
- extracting the board position to the FEN format,
- resetting the board to the starting position,
- the internal representation of a chess game using a tree-like or graph structure, allowing for handling alternative sequences of moves deviating from the main line. A crucial feature when analyzing top-level games,
- fast parsing of PGN files,
- communication with UCI compatible chess engines,

⁶ International Chess Federation

⁷ Universal Chess Interface (<https://www.shredderchess.com/chess-features/uci-universal-chess-interface.html>)

⁸ Standard (Short) Algebraic Notation, a FIDE standard for describing moves (<https://handbook.fide.com/chapter/E012018>)

⁹ Forsyth–Edwards Notation, a common format of representing a board position (https://en.wikipedia.org/wiki/Forsyth%E2%80%93Edwards_Notation)

- querying online Syzygy tablebase API,
- detection of absolute pins,
- detection of checkmates,
- detection of draws by insufficient material,
- detection of checks and attacks,
- counting number of moves without pushing the pawns and without captures for the fifty-move rule,
- detecting threefold and fivefold repetitions.

On top of the preceding requirements, the final product takes advantage of multiple threads available in modern CPUs, allowing games in a PGN file to be parsed in parallel, even further increasing the parsing speed and displaying a true dedication to boost performance.

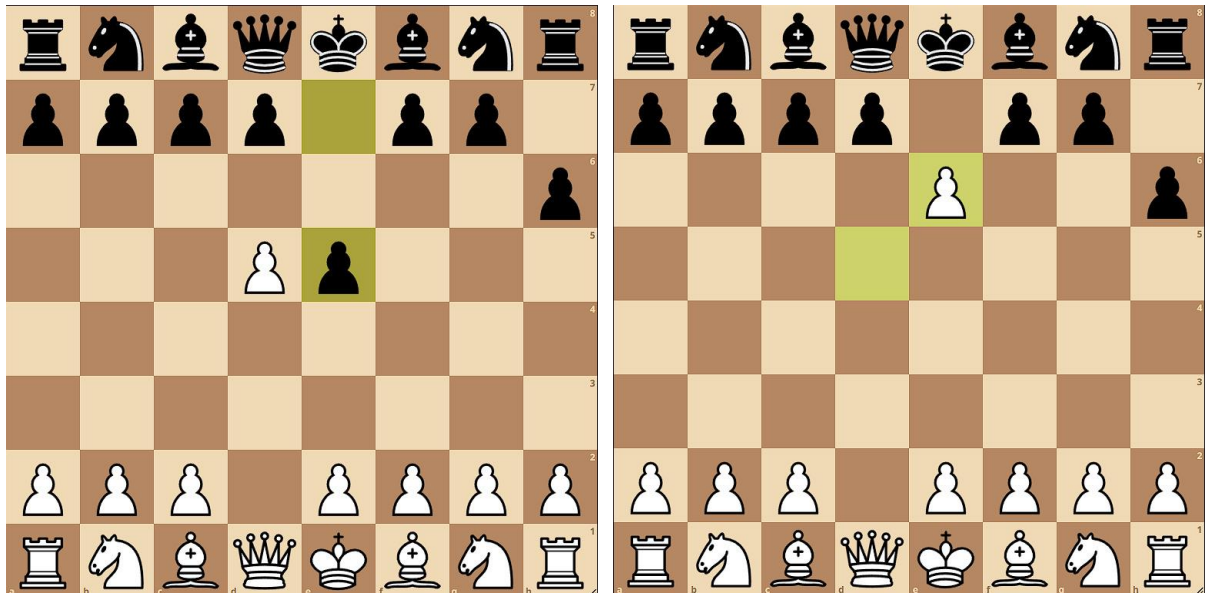


Figure 3. En-passant capture. A move possible only right after opponent's double pawn move. (<https://lichess.org/analysis>)

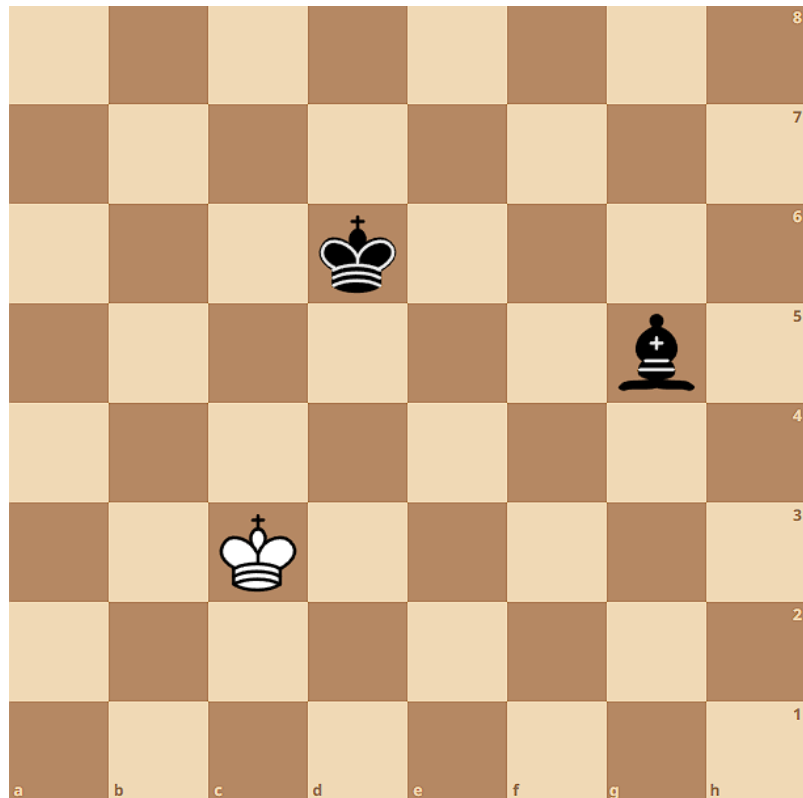


Figure 4. Draw by insufficient material. Neither side has the resources to deliver checkmate. (<https://lichess.org/analysis>)



Figure 5. Knight on c3 cannot move due to absolute pin. (<https://lichess.org/analysis>)

3. Computer Chess

3.1. General issue

Chess traditionally is a board game. Its core ruleset had emerged years before the computer revolution took place. Unremarkably the rules are very easy to be expressed verbally, using generalizations and loose statements. For example, to explain the movement of a bishop one can merely state that it moves diagonally. Restrictions to that movement include moving past other pieces, capturing the pieces of the same color, or making such a move that would result in the king of the same color being checked. Such restrictions are universal for other pieces and frankly very intuitive for humans. So intuitive in fact, that some chess tutorials entirely skip these details for simplicity. Consider this explanation.

Bishop

The bishop can move any number of squares diagonally - this means it always moves along the diagonal line of squares matching the current colour of its square. This means that each player begins the game with one bishop that can move on each colour. A bishop cannot move horizontally or vertically for any reason. It cannot move through pieces of the same colour, and captures a piece of the opposite colour by moving onto its square [11].

It is not precise. The author allocates the entire sentence of this short description to stress that a bishop can only move on a color it started with when the preceding sentence contains the exact same information. It states that it cannot move through pieces of the same color, when in fact it cannot move through pieces of any color. It dedicates another sentence to explain that a bishop cannot move horizontally or vertically but does not comprise one explaining that it cannot move for example 4 squares up and 3 squares right. The entire article never mentions that a move that would result in the player's own king being in check is illegal or that castling is illegal when a square between the king's initial and ending position is attacked by the opponent's piece. However, it is not a bad article. It is excellent at explaining basic concepts to new players shortly and comprehensively, that does not offend readers' intelligence.

In contrast, a definition like this, on its own, is ineffective in computer chess. Computers are not like humans. They are not insightful, do not have prior experience or knowledge unless provided, they are not eager to learn. Computers do not understand what it means to move bishop diagonally on the board. They do not understand the meaning behind the words bishop, diagonally, board, or even move at all. In general, computers only perform operations from a narrow set of instructions. In order to implement chess on computers, the mentioned notions must be ultimately expressed using the numerical representations and sequences of instructions solely from that set.

3.2. Game Representation

To differentiate an exclusive chess position, it is sufficient to consider the subsequent pieces of information:

- positions of all pieces on the board,
- which side to move (black or white),
- whether the king or rooks have moved (determines the castling rights).
- whether the last move enables en-passant capture (represented by a single square),
- number of consecutive non-zeroing¹⁰ moves for the fifty-move rule [2].

Many sensible approaches to represent a game of chess exist. The most common solutions are piece centric, square centric, or a combination of both. Piece centric manner focuses on storing pieces in a data structure of choice with the associated information about the occupied square, whereas square centric representations store squares and assign pieces to them. The simplest, array-based approaches have their pitfalls. Piece centric require more calculation when searching for the relationship of pieces and square centric are less efficient in generating moves [12] [13].

This paper concentrates on a more sophisticated solution as used in python-chess. A bitboard type is a piece-centric board representation using one bit for each square of the chessboard [14]. Bitboards aim to represent only the position of the pieces. All the remaining information is trivial to implement in the form of simple variables and optimizing this is counterproductive as there is little room for improvement starting from the most basic approach.

¹⁰ A non-zeroing is a move that does not push pawn and does not capture.

56	57	58	59	60	61	62	63
48	49	50	51	52	53	54	55
40	41	42	43	44	45	46	47
32	33	34	35	36	37	38	39
24	25	26	27	28	29	30	31
16	17	18	19	20	21	22	23
8	9	10	11	12	13	14	15
0	1	2	3	4	5	6	7

Figure 6. Chess bitboard

Bitboards, chess, and modern computers are a perfect match as chess board contains 64 squares, bitboards require one bit per square and recently computers switched to 64-bit architecture, in which the registers, a type of memory providing the fastest possible access, can store 64 bits of information each and thus a single register can hold an entire bitboard. It is an ideal scenario allowing for efficient bitwise operations [15]. Each bitboard describes the position of a single piece type or can be used in different schemes, for example storing the squares attacked by a particular piece or storing squares available for one. To represent a single chess position using bitboards two particular ways emerge. The first one is to create a bitboard for each piece type of each color. The second one is to create a bitboard for each piece type mutual for both colors and include companion bitboards to store a color that a square is occupied by. Rust Chess uses the latter.

The costs and benefits of bitboard include:

- bitboards may be used to represent complex conjugation of other bitboards,
- bitboards alone do not solve all problems of piece-centric representations, and complementary piece lists are necessary to quickly access information about relationships between pieces,
- bitboards can utilize bitwise instructions to make utterly fast calculations concerning all squares at once, in parallel,
- to store bitboards on 32-bit processors two registers are required, and the operations on bitboards may be split to two or more and take more CPU cycles. As most modern processors are now 64-bit, this point is becoming increasingly moot,

- bitboards reliance on bitwise operations may require use of hardware instructions exclusive for given architecture and manufacturer for optimal performance [16].

56	57	58	59	60	61	62	63
48	49	50	51	52	53	54	55
40	41	42	43	44	45	46	47
32	33	34	35	36	37	38	39
24	25	26	27	28	29	30	31
16	17	18	19	20	21	22	23
8	9	10	11	12	13	14	15
0	1	2	3	4	5	6	7

Figure 7. A bitboard storing occupied squares in the starting position

56	57	58	59	60	61	62	63
48	49	50	51	52	53	54	55
40	41	42	43	44	45	46	47
32	33	34	35	36	37	38	39
24	25	26	27	28	29	30	31
16	17	18	19	20	21	22	23
8	9	10	11	12	13	14	15
0	1	2	3	4	5	6	7

Figure 8. A bitboard storing position of knights in the starting position

56	57	58	59	60	61	62	63
48	49	50	51	52	53	54	55
40	41	42	43	44	45	46	47
32	33	34	35	36	37	38	39
24	25	26	27	28	29	30	31
16	17	18	19	20	21	22	23
8	9	10	11	12	13	14	15
0	1	2	3	4	5	6	7

Figure 9. A bitboard storing squares available for the undisrupted rook on the A1 square

56	57	58	59	60	61	62	63
48	49	50	51	52	53	54	55
40	41	42	43	44	45	46	47
32	33	34	35	36	37	38	39
24	25	26	27	28	29	30	31
16	17	18	19	20	21	22	23
8	9	10	11	12	13	14	15
0	1	2	3	4	5	6	7

Figure 10. A bitboard storing squares occupied by white pieces in the starting position

Figures 5-8 illustrate sample use cases for bitboards. The highlighted tiles stand for set bits of the bitboard and greyed-out tiles stand for unset bits.

3.3. Board, Pieces, Moves, Squares, and Colors.

As previously stated, the board is represented by a handful of bitboards. A piece, depending on the context in which used, can be expressed as either a set bit inside a bitboard or an enumeration. To differentiate colors only one bit of information is required, so a single Boolean value is used. Squares are implemented as a number between 0 and 63 inclusive. To describe a move three pieces of information are needed: which piece to move (or starting square), which square to move to, and an additional clue on what piece type to choose in the case of pawn promotion, also an enumeration. In the case of castling, it is sufficient to include the king's starting and resulting positions by virtue of a king being able to move only one square at a time, with exception of castling when it moves precisely two or three squares.

3.4. Move Generation

To generate legal moves, it is useful to incorporate a couple of intermediate steps first. Initially, a set of squares (later called an attack table) is created that a piece of consideration, standing on a specific square, unobstructed, can move to by the rules of chess. It is possible to calculate this dynamically, when needed, but it is more advantageous, in terms of performance, to calculate it ahead of time and store this information. These moves are saved as an array containing a bitboard with all the possible standard moves a piece can make from a specific square (often castling or pawn moves are omitted). The square represented as a number serves as an index of this array and each piece type has a dedicated array. The next step is to filter these possibilities, by excluding moves that go through pieces, creating so called attack masks, used to generate a list of pseudo-legal moves.

Pseudo-legal moves are all moves that follow the basic chess rules but can leave the king in check and castle while in check or over a checked square. All moves that capture the pieces of the same color, jump through other pieces (except for knights), etc. are not considered pseudo-legal [17].

The reason to generate pseudo-legal moves at all is that legality of a move is deeply conditional and incorporating such checks into the generator itself would be error-prone, hard to debug, and create avoidable turmoil. The process of filtering can be illustrated with the following algorithm:

```

00: FUNCTION generatePseudoLegal()
01:   color:= DETERMINE color to play
02:   pieces:= BITBOARD_OCCUPIED[color]
03:   opponentPieces:= BITBOARD_OCCUPIED[!color]
04:   nonPawns:= pieces & !BITBOARD_PAWNS
05:   # yield non-pawn pieces moves
06:   FOR each fromSquare in nonPawns
07:     targets:= ATTACK_MASKS[fromSquare] & !pieces
08:     FOR each toSquare in targets
09:       YIELD Move(fromSquare, toSquare)
10:     END FOR
11:   END FOR
12:   # yield castling moves
13:   YIELD generateCastlingMoves()
14:   pawns:= pieces & BITBOARD_PAWNS
15:   # yield pawn attacks
16:   FOR fromSquare in pawns
17:     targets:= PAWN_ATTACKS[color][fromSquare] & opponentPieces
18:     FOR toSquare in targets
19:       IF toSquare is at first or last rank
20:         FOR promotionPiece in [QUEEN, ROOK, BISHOP, KNIGHT]
21:           YIELD Move(fromSquare, toSquare, promotionPiece)
22:         END FOR
23:       END IF
24:       ELSE
25:         YIELD Move(fromSquare, toSquare)
26:       END ELSE
27:     END FOR
28:   END FOR
29:   # one square up means a bigger column number for white and smaller for black
30:   # one square down means the opposite
31:   singlePawnMoves:= CALCULATE empty squares one square up from all pawns
32:   doublePawnMoves:=
33:     CALCULATE empty squares one square up from singlePawnMoves on the
34:     3rd rank for WHITE and 5th rank for black
35:   # yield regular pawn moves
36:   FOR toSquare in singlePawnMoves
37:     fromSquare = CALCULATE a square one square down from toSquare
38:     IF toSquare is on the 1st or 8th rank
39:       FOR promotionPiece in [QUEEN, ROOK, BISHOP, KNIGHT]
40:         YIELD Move(fromSquare, toSquare, promotionPiece)
41:       END FOR
42:     END IF
43:     ELSE
44:       YIELD Move(fromSquare, toSquare)
45:     END ELSE
46:   END FOR
47:   # yield double pawn moves
48:   FOR toSquare in doublePawnMoves
49:     fromSquare = CALCULATE a square two squares down from toSquare
50:     YIELD Move(fromSquare, toSquare)
51:   END FOR
52:   # yield en-passant captures
53:   IF EP_SQUARE != NONE
54:     YIELD generatePseudoLegalEnPassant()
55:   END IF
56: END FUNCTION

```

The algorithm makes generating pseudo-legal moves simple by considering diverse move types separately. Functions `generatePseudoLegalEnPassant` and `generateCastlingMoves` are severely conditional and unraveling them would cause adverse effects on pseudo-code clarity but implementing them in code is simple because they follow the same patterns as generating regular pawn moves (calculations on bitboards containing the possibilities and branching).

To generate legal moves from the pseudo-legal moves one could just make the move, test if the king is in check (by inspecting the attack masks of the opponent's pieces) or if the castling is legal, and undo the move, keeping only the moves that pass the test. This is, however, possible only if the program supports the unmaking of moves. Please note it is less efficient to make and unmake a move than to entirely skip it.

4. Implementation

4.1. Tools

To hasten the development of the library, a selection of specialized tools, beyond just the integrated development environment, has been used. They did not impact the result in any way, besides obtaining it faster and with less struggle.

4.1.1. Bitboard Editor

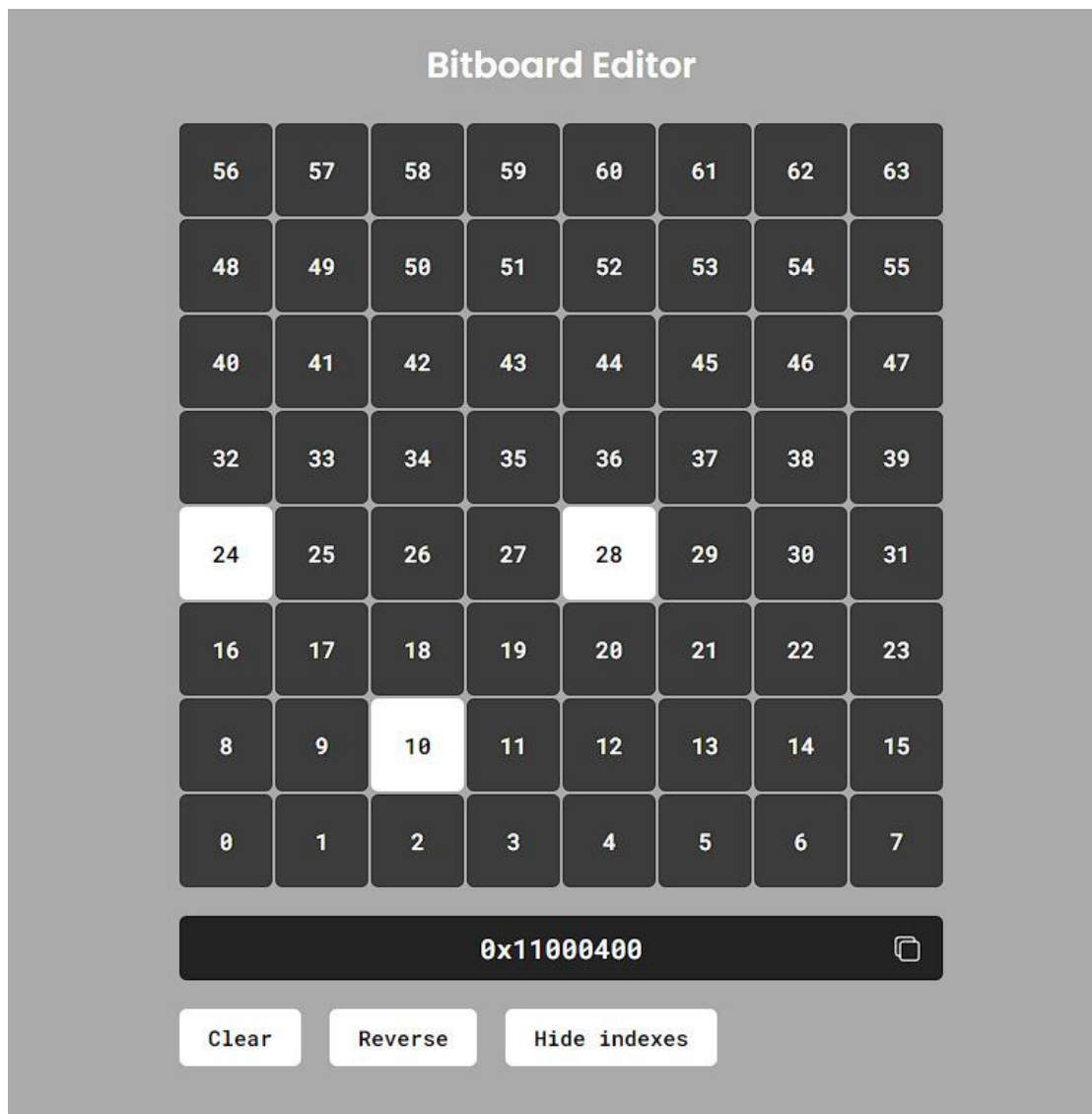


Figure 11. Bitboard Editor (<https://bitboard-editor.vercel.app/>)

The Bitboard editor is a straightforward web application for the purpose of interactively displaying bitboards. It allows to select tiles and produces an integer in the hexadecimal numeral system and vice versa. It was crucial while debugging the move generation as well as the parsing part of the library. For this scheme, a program is halted near the place where an issue occurs, the problematic bitboard is extracted and then pasted into the editor which highlights the tiles to visualize it, and then the programmer can select or unselect tiles to receive an integer representing the position that is expected in the program.

4.1.2. Pyrs, a Python to Rust transpiler

A Transpiler is a set of tools that take the source code developed in a high-level programming language (source language), and after a transpilation process, it generates a translated source code written in another programming language (target language), that is syntactically equivalent [18]

Pyrs¹¹ converts Python code to Rust code. It is far from a perfect tool, as it not only does not include a full feature set of neither language (rather a greatly limited subset), but the code it should be able to process often yields a wrong result (for example it notoriously translates the “not” keyword to “~” operator in contexts where “!” should be used). However, it is useful in translating certain simple snippets of repetitive code, and the incorrectness can usually be conveniently rectified at once by an advanced text editor such as Vim¹². Given a large amount of Python code in python-chess, normally resulting in an even larger amount of Rust code, the tool has allowed developing the library faster, despite necessary fixes and inspection.

```
from chain.block import Block

class Genesis(Block):
    def __init__(self, creation_time):
        self.timestamp = creation_time
        self.prev_hashes = []
        self.precalculated_genesis_hash = Block.get_hash(self)

    def get_hash(self):
        return self.precalculated_genesis_hash
```

```
use chain::block::Block;

struct Genesis {
    timestamp: ST0,
    prev_hashes: ST1,
    precalculated_genesis_hash: ST2,
}

impl Genesis {
    fn __init__<T0>(&self, creation_time: T0) {
        self.timestamp = creation_time;
        self.prev_hashes = vec![];
        self.precalculated_genesis_hash = Block::get_hash(self);
    }
    fn get_hash<RT>(&self) -> RT {
        return self.precalculated_genesis_hash;
    }
}
```

¹¹ Pyrs (<https://github.com/konchunas/pyrs>)

¹² Vim, a highly configurable text editor (<https://www.vim.org/>)

Figure 12. Example Python class to transpile

Figure 13. Resulting Rust code

```
if __name__ == "__main__":
    numbers = [1,5,10]
    multiplied = list(map(lambda num: num*3, numbers))

    comprehended = [n - 5 for n in multiplied if n != 3]

    print("result is", comprehended)
```

Figure 14. More complex Python example

```
fn main() {
    let mut numbers = vec![1, 5, 10];
    let multiplied = numbers.iter().map(|num| num * 3).collect::<Vec<_>>();
    let comprehended = multiplied
        .iter()
        .cloned()
        .filter(|&n| n != 3)
        .map(|n| n - 5)
        .collect::<Vec<_>>();
    println!("{:?} {:?}", "result is", comprehended);
}
```

Figure 15. Resulting Rust code

4.1.3. Windows Performance Analyzer, Windows Performance Recorder

To analyze which parts of a library require a special amount of attention, a set of programs from Windows Assessment and Deployment Kit were used. Windows Performance Recorder performs profiling on the running processes. Windows Performance Analyzer allows to filter through the results and load additional debug information to display detailed information about function calls. This is especially important because it provides the programmer with data about which functions are called most often, and what is the time duration of their execution. Knowing this, it is considerably easier to spot a place of potential improvements or a faulty algorithm.

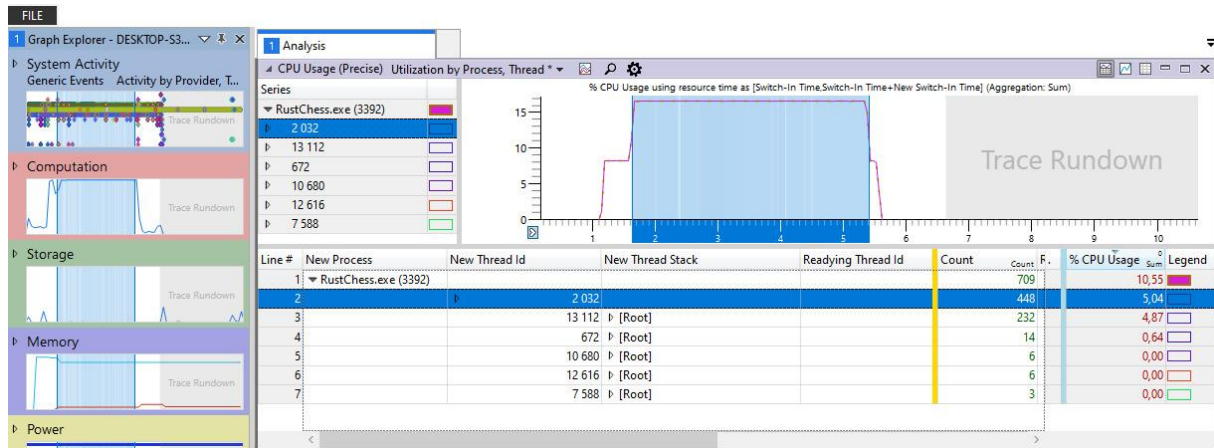


Figure 16. Windows Performance Analyzer GUI

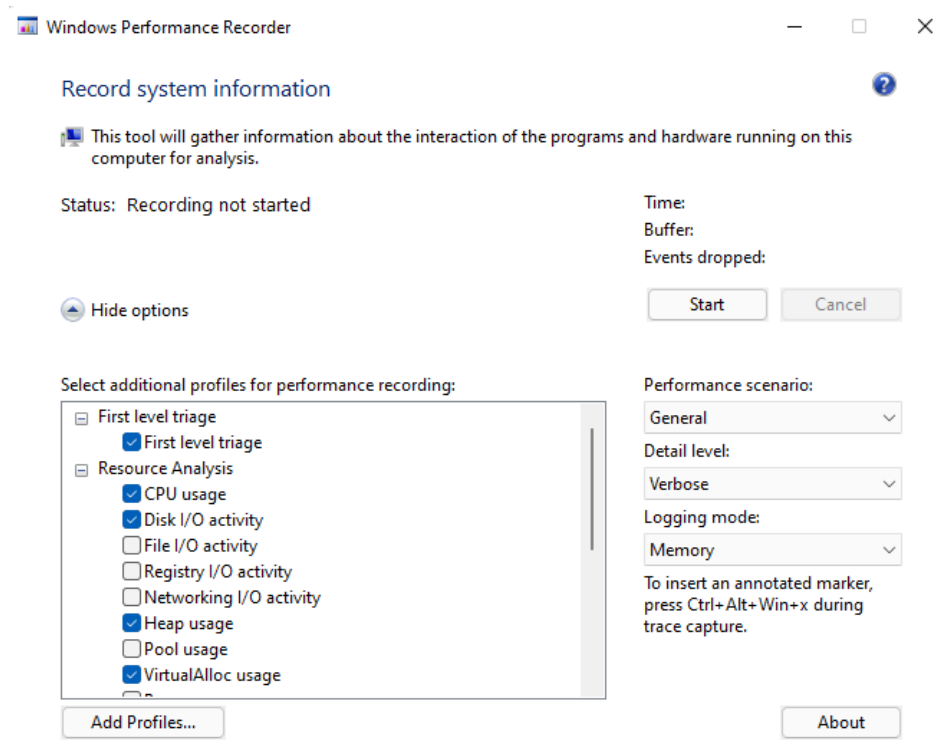


Figure 17. Windows Performance Recorder GUI

4.2. Design patterns

Some functionality of python-chess, designed with an intent to be expressed in Python, was troublesome to implement in Rust since if it were created with Rust in mind, the code arrangement would be vastly diverse and one of the criteria of this library was to keep it similar. This subsection showcases the solutions to these efforts.

4.2.1. Generators

Rust language does provide an iterator interface and iterators are widely used in Rust programs; however, it does not allow to generate iterators the way they are generated in Python with a `yield` keyword.

```
def scan_forward(bb: Bitboard) -> Iterator[Square]:
    while bb:
        r = bb & -bb
        yield r.bit_length() - 1
        bb ^= r
```

Figure 18. A Python generator (https://github.com/niklasf/python-chess/blob/master/chess/__init__.py)

The function included in Figure 18 each call yields the position of the next least significant bit of a bitboard (for example the first, then the second, and so on, pawn in the pawns bitboard). It does so very efficiently by skipping all the empty squares, with the help of bitwise operations. The implementation of a generator pattern is clean and compact. A direct realization of this pattern in Rust would result in messy and extensive code, that would have to be duplicated for each data type. Rust Chess uses a declarative macro¹³ based on a modified `gen_iter`¹⁴ crate¹⁵ to execute this design.

```
pub fn scan_forward(mut bb: Bitboard) -> impl Iterator<Item = Square> {
    gen_iter!({
        let mut r;
        while bb != 0 {
            r = bb & bb.wrapping_neg();
            yield (bit_length(r) - 1) as Square;
            bb ^= r;
        }
    })
}
```

Figure 19. A Rust generator using `gen_iter`

¹³ In Rust macros allow to generate code through patterns (<https://doc.Rust-lang.org/Rust-by-example/macros.html>)

¹⁴ `gen-iter` crate (<https://github.com/tinaun/gen-iter>)

¹⁵ Crate is a name for a binary package or library in Rust (<https://doc.Rust-lang.org/book/ch07-01-packages-and-crates.html>)

4.2.2 Inheritance

Whether Rust is an object-oriented language is disputable. The fact that Rust does not have an inheritance mechanism is certain. It is an intentional choice made by the creators to help keep the objects encapsulated. This follows the composition over inheritance principle which is considered beneficial for scalable projects [19]. The python-chess utilizes the inheritance in several parts of the program, but the most notable example is related to the class Board and its parent class BaseBoard. BaseBoard class is responsible for representing and manipulating a chess position using bitboards and the Board class handles gameplay-related information (color to move, castling rights, etc.). These classes do not share the same purpose or contain similar functionality, so it would not make sense to relate them with interfaces. After much consideration, BaseBoard was chosen to be a field in the Board struct. This way a program structure remains intact, but it requires specifying whether a BaseBoard or Board function or field is used e.g. for a function being part of BaseBoard, when handling Board instance, instead of “board.is_pinned(color,square)” a programmer needs to type “board.baseboard.is_pinned(color,square)”.

```
pub struct Board {  
    pub baseboard: BaseBoard,  
    pub ep_square: Option<Square>,  
    pub move_stack: Vec<Move>,  
    pub stack: Vec<BoardState>,  
    pub turn: Color,  
    pub castling_rights: Bitboard,  
    pub halfmove_clock: u64,  
    pub fullmove_number: u64,  
}
```

Figure 20. A struct Board containing a BaseBoard field

4.3. PGN representation

PGN is a file standard designed for human readability and easy parsing. While it is truly intuitive for chess players, computer parsing is more complicated. The major obstacle for computers is the use of SAN format for moves representation. Consider the position below.

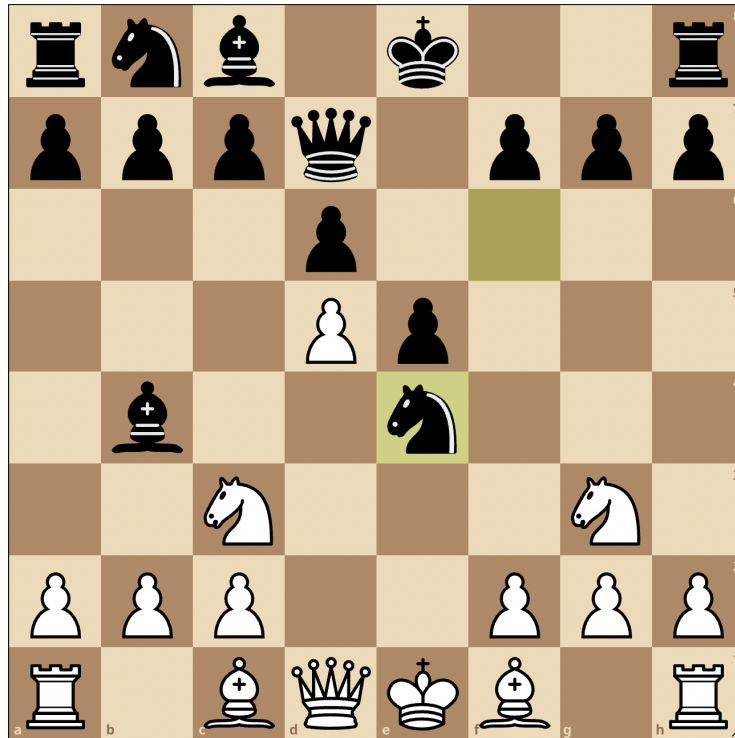


Figure 21. Exemplary edge case chess position

Assume that it is white's turn, and the intended move is capturing the knight on e4 with the knight on g3. Using the UCI format, the move is encoded as "g3e4". It is explicit and requires only checking if the move is legal. SAN format, conversely, is ambiguous without context. The given move is encoded as "Nxe4". Most board representation approaches would allow to quickly find the positions of all the white knights. Next, the question is which knight attacks the provided square. In this example both knights do. Normally, when there is such uncertainty in SAN, complementary information is included, and the move would be described with "Ngxe4", where g stands for the file the chosen knight stands on. Yet, in this position, only one of the knights can legally capture the knight e4, because the remaining knight is absolutely pinned, hence the "Nxe4". SAN is greatly dependent on the background information and thus the process of parsing it requires substantial calculations.

```

#[derive(PartialEq)]
2 implementations
pub struct NodeBase {
    pub is_root: bool,
    pub parent: Option<NodeRef>,
    pub m: Option<Move>,
    pub variations: VecDeque<NodeRef>,
    pub comment: String,
    pub starting_comment: String,
    pub nags: HashSet<u64>,
}
type NodeRef = Rc<RefCell<NodeBase>>;
3 implementations
pub struct Node (
    pub NodeRef
);
0 implementations
pub struct Game {
    pub root: Node,
    pub headers: Headers
}

```

Figure 22. Data structures representing a PGN game

A given game is represented using a tree-like graph, where each node owns an additional reference to a parent. A structure of this kind is enforced by RAV (Recursive Annotation Variation), which is supported by the PGN format. RAV allows to include variations to the main line. In Rust, a resource cannot be mutably shared by default, and it prevents all instructions that cannot be statically deemed safe by the compiler. For this reason, the Nodes are wrapped with RefCell, which allows mutable access to a memory location, by dynamically checking the borrows¹⁶, and then wrapped with Rc, a reference-counting pointer, enabling the shared access to the RefCell memory location.

1.e4 c5 2.c3 (2.Nc3 e6 3.f4 (3.Nf3) (3.g3))

Figure 23. RAV in a PGN format

A game in this format is constructed using a design pattern called visitor. Visitor's appropriate method is called when an object is encountered in a PGN text being read and it can selectively choose to include information in a game representation.

¹⁶ Borrowing is a part of an ownership mechanism in Rust (<https://doc.rust-lang.org/book/ch04-01-what-is-ownership.html>)

```
pub struct GameBuilder {
    pub game: Game,
    variation_stack: Vec<NodeRef>,
    starting_comment: String,
    in_variation: bool
}
```

Figure 24. A representation of a visitor, parsing full PGN information

```
impl GameBuilder {
    fn new() -> GameBuilder { ...
    fn begin_game(&mut self) -> Option<SkipType>{ ...
    fn new_and_begin() -> GameBuilder { ...
    fn begin_headers(&self) -> Option<&Headers>{ ...
    fn visit_header(&mut self, tagname: &str, tagvalue: &str){ ...
    fn visit_nag(&self, nag: u64) -> Option<SkipType> { ...
    fn begin_variation(&mut self){ ...
    fn end_variation(&mut self) { ...
    fn visit_result(&mut self, result: &str) { ...
    fn visit_comment(&mut self, comment: &str){ ...
    fn visit_move(&mut self, _board: &Board, m: Move){ ...
    fn visit_board(&self, _board: &Board){ ...
    pub fn result(&self) -> &Game { ...
    pub fn parse_san(&self, board: &Board, san: &str) -> Move { ...
```

Figure 25. Set of visitor capabilities

4.4. Optimizations

Without a doubt, the most significant factor contributing to Rust Chess having better performance is the language itself, but to port a well-written program to a faster language is not enough. It is very straightforward to write a poor quality, underperforming program. To make use of the existing potential, measures that constitute that potential must be utilized.

4.4.1. Build Configuration

An obvious, but crucial and easy to forget detail is to use a release build and not the default debug build.

```
Finished dev [unoptimized + debuginfo] target(s) in 0.05s
Running `target\debug\RustChess.exe`
n: 12346, n/t: 3086
time elapsed: 636.7434ms
```

Figure 26. Exemplary parsing speed using debug build

```
Finished release [optimized + debuginfo] target(s) in 0.05s
Running `target\release\RustChess.exe`
n: 12346, n/t: 3086
time elapsed: 65.86ms
```

Figure 27. Exemplary parsing speed using a release build

The time difference is so profound that even during the development, release builds were usually used (with the debug information included), which resulted in longer compilation times but much faster execution times when dealing with large files. The biggest distinction between these profiles is that in a release build all compiler optimizations are enabled [20] [21].

Another option to consider is the link-time optimizations, which improve performance, once again increasing the compilation time and aborting instead of unwinding when the program panics. To take full advantage of frequent bitwise operations, CPU manufacturer-specific instructions may be enabled.

4.4.2 Reading a file

An uttermost mistake in the early stages of Rust Chess development was a suboptimal method of reading a file. It used to read a chunk of a file to a buffer as needed and perform parsing on the buffered data. This made an impression of a great alternative to reading a file line by line, which is a common and convenient approach but allocates a new string for every line. At the time the program used memory at least 4 times larger than the file size. The program used `BufReader` from the standard, stable library, thus a programming error was unlikely. It was a clear bottleneck, so several other choices were tried. Out of them, simply reading the whole file (that can reach orders of GBs) into the memory proved the fastest. The file is stored in a string, split into chunks containing one full game each, and passed to the parser. This single change alone contributed to the program becoming about 8 times faster and paradoxically using less memory. An undeniable drawback of this solution is that whether a file of a certain size can be parsed depends on the amount of available random-access memory. Files much bigger than available RAM need to be split into separate files.

4.4.3. Selecting the hash function

Inside the library, there are many instances of data collections using the hash function¹⁷. The said data collections are hashmaps and hashsets. Rust, by default, uses a cryptographic hash function, which provides better collision avoidance and resists certain types of attacks at the cost of speed [22]. In Rust Chess, security mechanisms are not important, and the mentioned data structures hold a rather moderate number of objects, so the collisions are unlikely (otherwise they could impact the performance). The default hash function was switched for a faster, non-cryptographic one¹⁸. It increased the speed of parsing by 5%-10%.

4.4.4. Optimizing individual snippets

A peculiar example of small optimizations was one regarding the `parse_square` function. It receives an input string slice containing the name of a square and returns an integer of type `Square`, which is an alias for an 8-bit number. This function is called every time a SAN format is parsed, so even minuscule changes in speed could impact the execution times considerably. Several approaches were considered. The original one, inscribed in Figure 28, converts a string slice to a `Chars` object, casts both characters to 8-bit numbers, and calculates the proper rank and file numbers through the help of `ASCII`¹⁹ properties. Then it returns the square calculated with the rank and file numbers. The conversion of a string slice to `Chars` object, according to Rust documentation, can be done in constant time, but disassembly of Rust Chess code shows that it was not. To combat this the function was replaced with a hashmap, whose keys contained all the names of squares, and the values held corresponding square numbers. This approach turned significantly slower than the prior, mainly due to using the default cryptographic hash function. After changing the hash function, the hashmap approach was tried again, this time more comparable, but still slower than calculating the value. Another tested way was extracting the square number using lookup tables²⁰. A large (2^{15} B = 32768B, cells have the size of 1B), mostly empty array was constructed, in which specific indices contained corresponding square numbers. This too was quite close, but still a little slower than calculating the number, as it required some precalculations of the index. It came to light that the original approach was not that slow after all.

¹⁷ Hash function (<https://mathworld.wolfram.com/HashFunction.html>)

¹⁸ Rust Chess uses AHash algorithm, which utilizes hardware AES instruction (<https://docs.rs/ahash/0.7.6/ahash>)

¹⁹ American Standard Code for Information Interchange (<https://en.wikipedia.org/wiki/ASCII>)

²⁰ Lookup table is an array that replaces runtime computation with a simpler array indexing operation (https://en.wikipedia.org/wiki/Lookup_table)

```
pub fn parse_square(name: &str) -> Square {
    let file: u8 = name.chars().nth(0).unwrap() as u8 - 'a' as u8;
    let rank: u8 = name.chars().nth(1).unwrap() as u8 - '1' as u8;
    return rank * 8 + file
}
```

Figure 28. *Parse_square* function of Rust Chess

4.5. Syzygy tablebases

Syzygy tablebases allow for quick lookup of a precalculated result of a game for positions containing 7 or less pieces (including kings), without castling rights. Probing of tablebases was implemented using requests to the Lichess HTTP API²¹. A choice to only probe online databases instead of probing a file read from disk as well (for up to 6-piece endings) was made solely because of time constraints on the thesis project. This functionality is added for convenience of a user who would like to manually probe the tablebase without using an engine. In this scenario the time overhead of making an online request is negligible. To make a request a user calls a `make_api_call` function and receives the optional `QueryResult` instance in result. `QueryResult` contains information about a winner (white, black or none), a depth to a zeroing move²² and a list of moves leading to the shortest forced mate, if possible, assuming the opposing color's perfect play.

```
pub struct QueryResult {
    dtz50: u32,
    mainline: Vec<Move>,
    winner: Option<bool>
}
```

Figure 29. *QueryResult* structure

²¹ Lila-tablebase (<https://github.com/lichess-org/lila-tablebase#http-api>)

²² A measure indicating in how many moves a zeroing move happens. A zeroing move resets the ply count related to 50-move rule.

```

pub fn make_api_call(fen: &str) -> Option<QueryResult>{
    let response: Response = match requests::get(
        url: format!("{}", fen), URL, str::replace(fen, " ", "_"))
    ) {
        Ok(r: Response) => {r},
        Err(e: Error) => {println!("bad request: {:?}", e); return None;}
    };
    let json: JsonValue = match response.json() {
        Ok(r: JsonValue) => {r},
        Err(e: Error) => {println!("response: ({})",
            std::str::from_utf8(response.content())
                .expect("error while converting Vec<u8> to string")); return None;}
    };
    match json["winner"].as_str() {
        Some(win: &str) => {
            match win {
                "w" => Some(QueryResult::get_decisive_result(dtz: json["dtz"].as_u32().unwrap(),
                    json_games: json["mainline"].members(), winner: Some(WHITE))),
                "b" => Some(QueryResult::get_decisive_result(dtz: json["dtz"].as_u32().unwrap(),
                    json_games: json["mainline"].members(), winner: Some(BLACK))),
                _ => None,
            }
        },
        None => Some(QueryResult::get_decisive_result(dtz: json["dtz"].as_u32().unwrap(),
            json_games: json["mainline"].members(), winner: None)),
    }
}

```

Figure 30. make_api_call function

Figure 31 shows two chess positions, one of which is winning for white, and the other is drawn. For human players, it is essentially impossible to evaluate some positions correctly and the chess engines usually calculate only up until some depth and can overlook certain moves. The endgame tablebases provide an answer, and they provide it fast. The position on the left is a forced mate in 549 moves and the position on the right is drawn. Of course, the winning position disregards the fifty-move rule. The drawn position is the 9th game of the 1890 World Championship match between Wilhelm Steinitz and Isidor Gunsberg. For over a hundred of years chess players debated whether the position was winning for black, leaning toward a positive answer [23]. The tablebases proved otherwise.

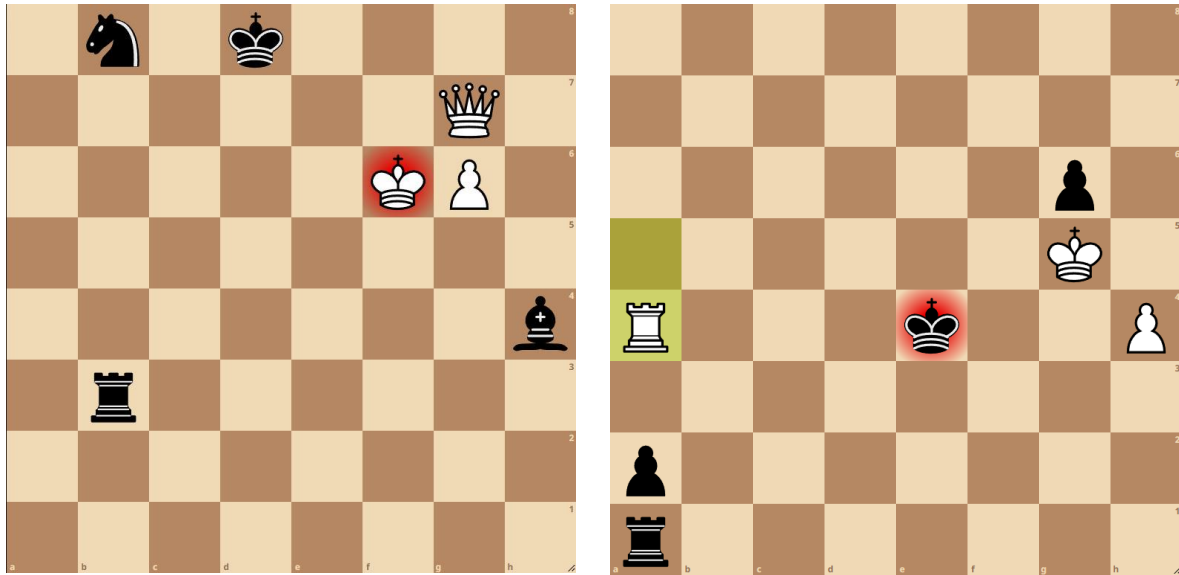


Figure 31. Example of a winning and drawn position (<https://lichess.org/analysis>)

4.6. UCI communication

Today, many open-source engines are available for chess players and programmers. The chess engine scene is dominated by a handful of extremely efficient engines and writing a new one matching their performance would be incredibly difficult. Universal Chess Interface allows for standardized communication with them, enabling the opportunity to incorporate them into other programs. Thanks to UCI, Rust Chess can provide the virtually best possible evaluation on the given machine. Information sent over UCI includes configuration and status data, selected position, and commands to start and stop searching. The number of options to configure depends on the chess engine used, but commonly include settings like the path to Syzygy or Nalimov tablebases, amount of memory to use, strength limit, or whether to use opening books.

```

let mut eng: Engine = Engine::new(
    engine_path: "stockfish/stockfish_14.1_win_x64_avx2.exe"
);
let mut game: Board = Board::new(fen: Some(STARTING_FEN));
let mut best_move: Move;
eng.start();
eng.set_position(STARTING_FEN);

for _ in 1..10 {
    best_move = eng.get_best_move(time_ms: 100);
    println!("move :{}", best_move);
    game.push(best_move);
    eng.set_position(
        &game.fen(promoted: false)
    );
}
println!("{}", game.baseboard.unicode(true, false, "."));

```

```

move :Move.from_uci(e2e4)
move :Move.from_uci(c7c5)
move :Move.from_uci(g1f3)
move :Move.from_uci(e7e6)
move :Move.from_uci(d2d4)
move :Move.from_uci(c5d4)
move :Move.from_uci(f3d4)
move :Move.from_uci(b8c6)
move :Move.from_uci(b1c3)

```

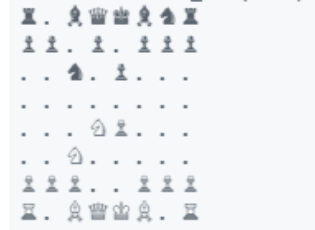


Figure 32. Rust Chess communication with an engine

5. Use cases

This chapter showcases a collection of potential adoptions of Rust Chess in practical applications. The range of capabilities is similar to that of python-chess, thus the types of applications in which the library may be used are virtually the same. The difference lies in the practicality of either library for the given purpose.

5.1 GUI chess game

The clearest and most straightforward application of Rust Chess is to use it in a computer application to play chess. The moves of the AI opponent would be generated by a chess engine, but the chess engine alone does not suffice. For example, Stockfish, one of the strongest and most popular chess engines, when provided with an illegal chess position, produces undefined behavior, which in the best-case results in a crash. To avoid this, a chess analysis library can be used to make sure only legal and correct positions are sent. In this scenario, Rust Chess could be used to:

- represent the internal chess board of the program,
- provide a representation of moves and position in common chess formats,
- generate legal moves to restrict a set of actions a user can make in GUI, or make the player lose a game when an illegal move is played,
- encode GUI events as chess moves and provide the GUI about chess events such as stalemate or checkmate,
- mediate in communication with a chess engine,
- counting the number of moves and providing the list of previous moves.

In essence, the library is capable of playing a game as is but lacks the proper abstractions and GUI to make the game interactive and enjoyable.

```
let mut game: Board = Board::new(fen: Some(STARTING_FEN));
game.push(game.parse_san("e4"));
game.push(game.parse_san("e5"));
game.push(game.parse_san("Qh5"));
game.push(game.parse_san("Nc6"));
game.push(game.parse_san("Bc4"));
game.push(game.parse_san("Nf6"));
game.push(game.parse_san("Qxf7"));
println!("is it checkmate?: {}", game.is_checkmate());
println!("{}", game.baseboard.unicode(!game.turn, false, "."));
```



Figure 33. Scholar's mate in Rust Chess

5.2 Chess game statistics

Fast PGN file parsing along with remaining chess features allows for extracting inconspicuous and very useful data from a large number of games. For example, a program to calculate and store FEN representations of each position in a game could be created. This additional information could be stored in a database, and a query to this database with chess position in FEN format would return a PGN game or a PGN game header. Then, another program or separate module of the same application could provide the user playing or analyzing the game with interesting information. The program could show percentages of how often a given position ends in a draw, win by white or win by black. It could list the names of chess grandmasters who played this position, tell whether a position is known to be drawn or winning by either side, and even show precalculated engine evaluation with best moves. In this scheme performance of the library is crucial and though python-chess theoretically could be used to create such a program, the amount of computation and parsing required is enormous, thus a database of sufficient size could not be created in a reasonable time.

5.3 Chess engine backend

Chess engines utilize multiple functions available in chess analysis libraries. To create a new chess engine from Rust Chess, an evaluation function would have to be implemented. As previously mentioned, creating a competitive engine is extremely hard, and if a new engine with the ability to face Stockfish emerges, it's very unlikely to use a chess analysis library of this kind. However, creating an engine for educational purposes certainly makes sense, and it could help programmers to focus on the evaluation function right away.

Another example is that when designing a chess position evaluation function e.g., with a deep neural network using reinforcement learning paradigm, recreating a board representation, and generating moves is an issue of a different domain, but these functions are very often needed during the development stage. Rust Chess could be used to check while in training, whether a function returns a legal move or to provide a different chess engine evaluation to see the progress in the early phases of learning.

6. Results

To make the results credible and comparable, a simple methodology to test the speed of execution was assembled. It assesses the libraries in a practical application, in a fashion a programmer is most likely to use. Programs tested for the PGN parsing speed take the following steps:

1. read a PGN file,
2. randomize the order of games,
3. start the timer,
4. parse PGN games,
5. stop the timer,
6. calculate the time difference
7. Repeat steps 1-6 10 times (in a loop).

Each library is tested with the same PGN files²³. The selected PGN files' sizes are representative of what most of the online sources provide (about 0.1-10 MB²⁴ or 150-15000 games) [24] [25]. Furthermore, the tests were repeated for two large datasets of 489 MB and 1702 MB to test performance in handling a huge number of games. The result is an average of 10 measurements for each file size (with exceptions). The test programs were run on the same hardware (Intel i5 10400f, 16GB of 3200MHz RAM, SATA III SSD), single-threaded unless explicitly stated. Between the tests, the disk cache was cleared, and the system was guaranteed to be free of any resource-hogging processes.

6.1. Python using the regular interpreter

Table 1 contains the results for the python-chess, using the official Python 3.9.6 interpreter. For practical reasons, the measurements for the largest of files (1702 MB) were omitted, and for the file of size 489 MB, the result is an average of 4, instead of 10 measurements. Full tests were not realized, because they were lingering. The average time of the latter – 948s – that is, almost 16 minutes, makes the use of the Python interpreter unfeasible for files this size.

²³ The files are Anand.pgn, Ashley.pgn, Averbakh.pgn, RuyLopezOther3.pgn [26], and selected sets of games from March 2015 on Lichess.org (<https://database.lichess.org/>)

²⁴ The units use SI prefixes, 1MB = 1 megabyte, not to be confused with the sizes shown by e.g., Windows Explorer, where 1MB = 1 mebibyte, or 1.048576 megabytes.

Size [MB]	Median time [ms]	Average time [ms]	Average speed [MB/s]
0.27	970.82	967.64	0.28
0.58	1993.52	1996.79	0.29
2.61	8962.07	9000.41	0.29
8.06	27461.62	27372.47	0.29
489.18	950528.76	948179.13	0.52

Table 1. python-chess [interpreter] average performance

To illustrate the average time throughout the repetitions (Figure 34) a 2.6 MB file is examined. The execution times were consistent with an average of 9.00s and a standard deviation equal to 0.084s.

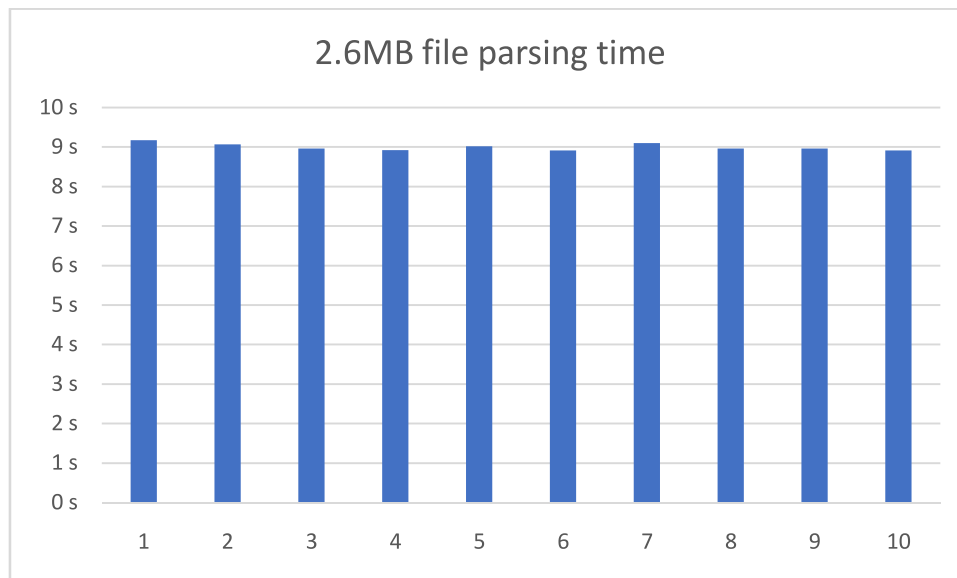


Figure 34. python-chess [interpreter] 2.6 MB file parsing time

6.2. Python, using PyPy

The results for python-chess using PyPy were somewhat surprising and intriguing. As with the tests using Python interpreter, the measurements for the largest of files were omitted and repeated 4 times for the 489 MB file.

Size [MB]	Median time [ms]	Average time [ms]	Average speed [MB/s]
0.27	745.67	1133.66	0.24
0.58	1437.76	1725.18	0.34
2.61	5965.42	6689.37	0.39
8.06	18366.49	19167.09	0.42
489.18	582508.00	588638.41	0.83

Table 2. *python-chess [PyPy] average performance*

PyPy was, on average, faster than the interpreter, but for some of the tries, it was significantly slower. The median time, especially for the smaller file sizes, is noticeably smaller than the average time. The 489 MB file took, on average, more than 588 seconds (9 minutes and 48 seconds) to parse, making the python-chess using PyPy impractical larger files.



Figure 35. *python-chess [PyPy] 2.6 MB file parsing time*

Figure 35 shows that for PyPy, the first couple of iterations were notably slower than the rest. It appears that PyPy uses adaptive mechanisms to improve performance in monotonous tasks, once it can predict the order of execution, based on collected data or optimizes frequently used parts of code, thanks to just-in-time compilation. Nevertheless, for the users of chess analysis software, it is hardly ever the subsequent speeds that matter. Even though the typical, as well as the average speed, is higher than that of interpreted Python, this and the inability to quickly parse large files make an impression of rust-chess using PyPy being the least sensible choice for most applications.

6.3. Rust Chess

Rust chess' results are exactly as expected. Quicker execution times allowed to perform full tests, including 10 iterations for both 489 MB and 1702 MB files. The results' variation is minor throughout the repetitions and overall speed, although increases with file size up until some point, is truly satisfying for all sizes.

Size [MB]	Median time [ms]	Average time [ms]	Average speed [MB/s]
0.27	3.11	3.26	83.15
0.58	6.47	6.90	83.89
2.61	28.09	28.09	92.88
8.06	80.02	79.95	100.79
489.18	5671.61	5650.84	86.57
1702.00	19989.75	19920.77	85.44

Table 3. Rust Chess average performance

The average and median time are very similar. The largest of files (1702 MB) took, on average, just below 20 seconds to parse. The average speed varied between the file sizes and peaked at 100.79 MB/s for the 8.06 MB file. The time duration throughout the repetitions remained fairly constant and did not indicate any distinct patterns.

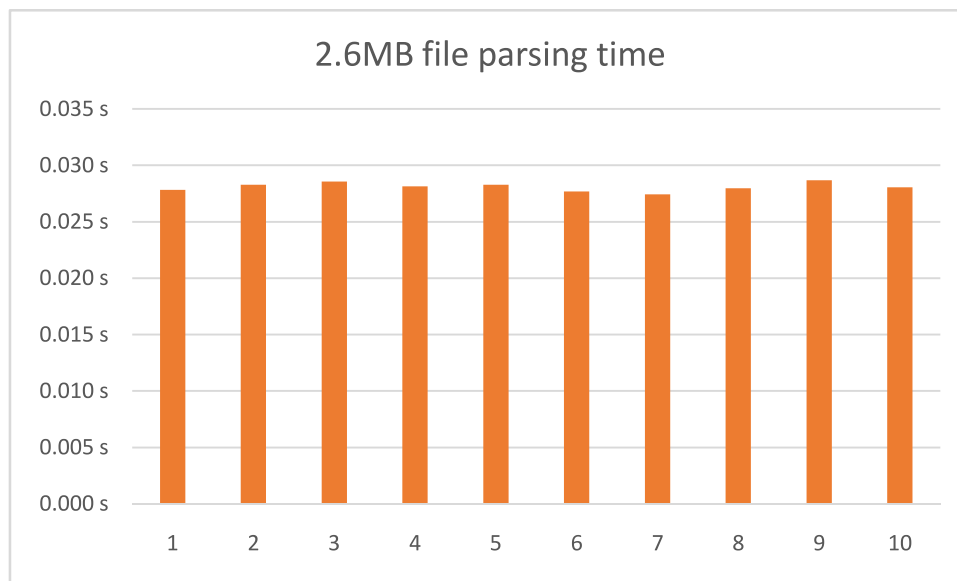


Figure 36. Rust Chess 2.6 MB file parsing time

6.4. Rust Chess, multi-threaded

On top of good single-threaded performance, Rust Chess allows parsing a PGN file in multiple threads, in parallel. Doing so, the average time is reduced even further. According to Amdahl's law, the theoretical limit to a speed increase, when the whole task is parallelizable, is the number of processor cores times the original speed.

$$Speedup = \frac{1}{\left((1 - P) + \frac{P}{N} \right)}$$

P = parallelizable fraction of a program, N = number of processors [26]

Parsing PGN files should approach this theoretical limit, as the amount of strictly serial operations necessary is minuscule. As of the time of writing this thesis, this is not true for Rust Chess. The performance of a single thread is likely to be throttled as soon as there is another thread running, and for a larger number of threads, it is certainly throttled, ultimately leading to even worse performance than running single-threaded, as the number of threads increases. The source of this behavior is undetected, the search for hotspots in CPU and Memory usage using a profiler did not yield explicit results. It is likely to be related to the memory bandwidth, but further investigation is needed to obtain a clear answer.

Still, the library does work correctly multi-threaded and for a small number of threads, it does work faster. Making it faster is the entire point of implementing multithreading. There is not an obstacle to using it multi-threaded as is.

No. threads	Average time [s]	Average speed [MB/s]	First run time [s]	First run speed [MB/s]
1	5.65	86.55	5.69	85.97
2	4.37	111.93	3.46	141.49
3	4.22	115.91	3.37	145.27
4	4.22	115.96	3.28	148.90
6	4.75	102.91	4.05	120.82

Table 4. Rust Chess 489 MB file average performance per no. threads

Table 4 compares average performance to first run performance. First run speeds for multiple threads are much higher than the average, in contrast to deviations occurring in single-threaded tests using PyPy. A fast first run speed is of higher utility than a fast speed of subsequent runs, but it reveals the problem of throttling. The reason for this behavior is likely to be the same as for diminishing speed when the number of threads increases.

It is important to note that neither Python interpreter nor PyPy supports true multithreading because of implementing a mechanism called GIL (Global Interpreter Lock), which prevents running threads in parallel as a consequence of their execution of garbage collection mechanism [8] [27]. Python supports parallelism by using subprocesses via multiprocessing²⁵ library. Each process owns its own memory; thus, GIL is bypassed, and processes can run on multiple CPUs. PyPy too supports spawning processes, but via multiprocess²⁶ library.

6.5. Comparison

Figure 37 compares the median speed of parsing various PGN files by Rust Chess and python-chess using PyPy and Python interpreter. Even though the chart is logarithmic, the difference is very pronounced. For 0.27 MB file, Rust Chess is over 300 times quicker than interpreted python-chess and it parses the 1702 MB file in a time comparable to one needed for parsing the 8.06 MB file by its direct competitors. These results are a true justification for Rust Chess development.

²⁵ Multiprocessing library (<https://docs.python.org/3/library/multiprocessing.html>)

²⁶ Multiprocess library for PyPy (<https://pypi.org/project/multiprocess/>)

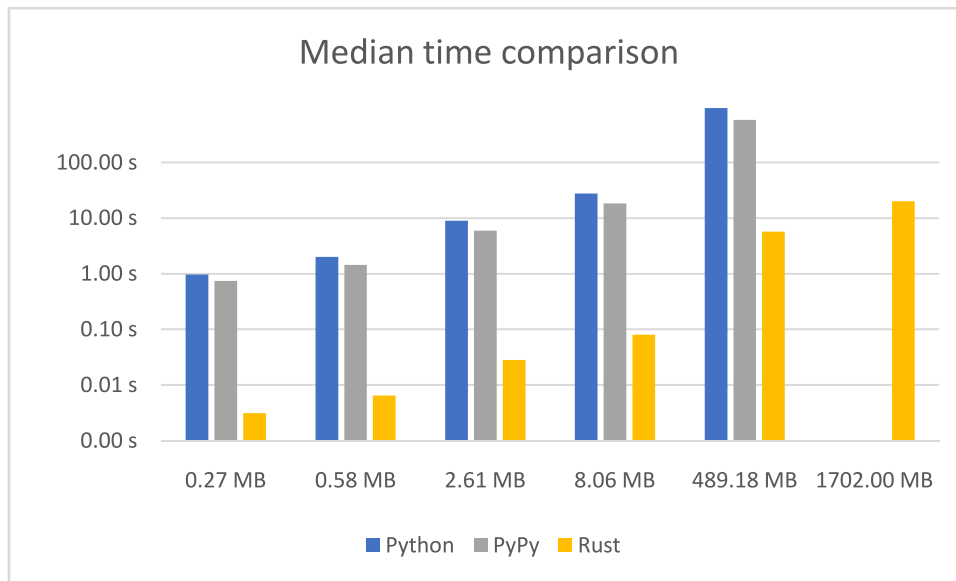


Figure 37. Median time of parsing comparison. Note the logarithmic scale on the vertical axis

6.6. Game speed analysis

Next, the performance in a test aimed at representing a practical application was measured. The test programs create a board instance, and alternately calculate all legal moves and make moves from an exemplary game. This is explained with the following algorithm:

1. start the timer,
2. create a board instance,
3. set the starting position,
4. calculate all legal moves,
5. make another move,
6. repeat steps 4-5 until the end of a game,
7. repeat steps 2-6 10000 times,
8. stop the timer,
9. calculate the time difference.

The presented values are the median time of 10 manual executions of the test programs. The games were taken from the 8.06 MB file, and games were prepared (represented as a list of move instances) before starting the timer, and thus this is not included in results. Rust Chess is more than 30 times faster than python-chess (using Python as an interpreter) in this scenario.

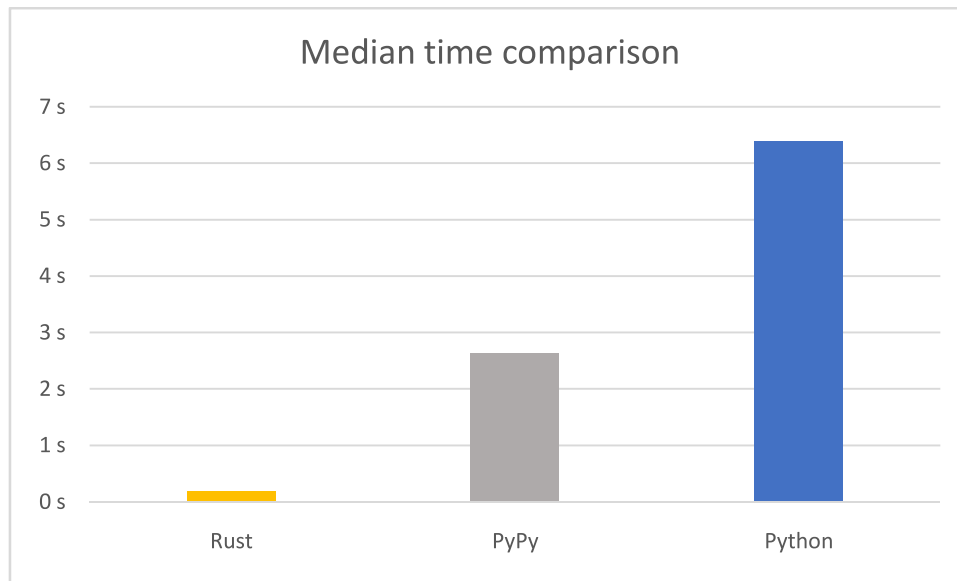


Figure 38. Comparison of median times in a test simulating practical use

6.7. Other chess analysis software

Naturally, Rust Chess was intended to be compared not only to python-chess but other chess analysis libraries as well. Searching for a library of this kind written in a language whose speed is comparable to that of Rust proved difficult. There are projects written in languages such as C#, Java but these normally use virtual machines and just-in-time compilation, which usually make the execution time slower (compared to regular compilation), so the results would be once again language-related and would not put the quality of implementation to test.

The first found mention of a chess analysis program written in C++, a language comparable with Rust in terms of speed, was Jerry. The author's blog post mentions that his optimizations result in parsing a file "millionbase-2.22.pgn" in 4 minutes and 12 seconds [28]. Jerry's new releases, however, are reimplemented in Java, and the quoted file is no longer easily available on the Internet, but a research of the web archives suggests that a file contained 2.22 million games [29]. In contrast, the 1.7GB file used for testing Rust Chess contains 1.743 million games (and single-threaded Rust parses it in less than 20 seconds on the test machine).

The most interesting finding was undoubtedly `cpp-chess`²⁷. It may be considered as Rust Chess' main rival because it is a sister project. Similarly, it is a chess analysis library, based on `python-chess` and rewritten in a fast language. The project was publicly shared in July 2021, it was discovered by me after the start of the development of Rust Chess, and it is still in progress. Unfortunately, it does not implement PGN parsing, so comparison of this aspect is impossible. Furthermore, the code pulled from the most recent commits does not compile and generates numerous errors.

The last found project was a full analysis program written in C++, called Cute Chess²⁸. It does work as expected, and the source code is available on the Internet, so it could be recompiled with minor changes to measure the performance. As it is a chess program and not a library, the methodology used for Python and Rust libraries was impractical, as it would require deeply inspecting the inner workings of the program, modifying it, and writing a substantial amount of new code. Instead, only the PGN parsing speed was measured, and it was not repeated 10 times in a loop, but was repeated 10 times manually, on a new, also manually randomized file. Figure 39 compares these results to single-threaded Rust Chess, as Cute Chess only supports parsing PGN in a single thread.

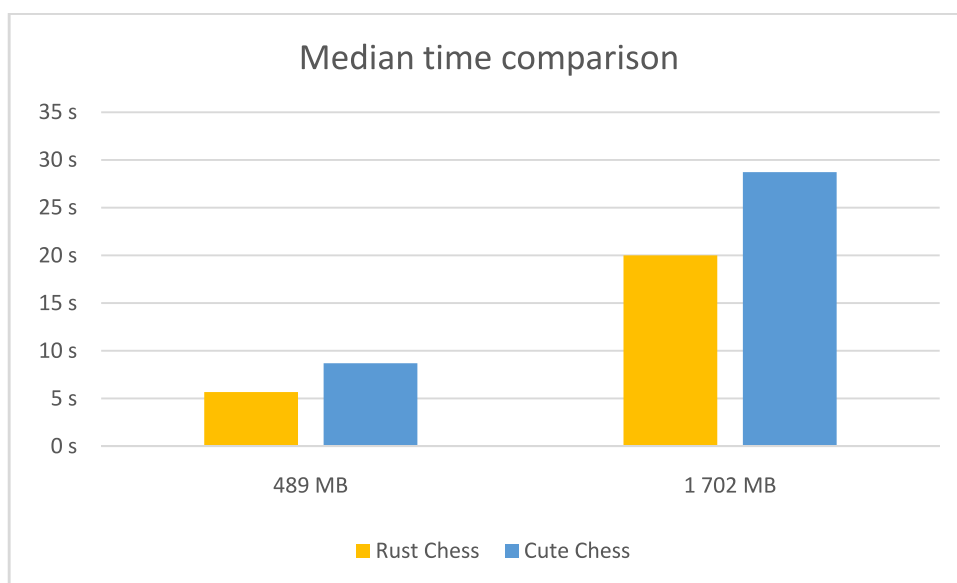


Figure 39. Rust Chess compared to Cute Chess

Even against a fair match, Rust Chess prevails. This illustrates that the library is a successful affair, and its performance is adequate for use in serious chess analysis software.

²⁷ `Cpp-chess` (<https://github.com/pjpuzzler/cpp-chess>)

²⁸ Cute Chess (<https://cutechess.com/>)

7. Conclusions

To summarize, the thesis project was successfully finalized, and the results are quite satisfying. The performance of Rust Chess exceeded initial expectations by parsing PGN files more than two orders of magnitude faster than its predecessor, python-chess. All posed requirements were met but for some parts of the library only the core functionality is present. Nevertheless, the disregarded features were chosen prudently, as to fit in the time limit but maximize the utility.

The thesis topic was at the heart of Computer Science and came to be genuinely didactic. To implement some functionality in another language, and to form a well-performing behavior required a great deal of research, plenty of consideration, and especially testing, measuring, and time.

Of course, there is always room for improvement and new features, but with software, it is rarely the case that a product is complete. Many programs are in development for decades now, and there just comes a time to conclude existing functionality and publish. Hopefully, this is not the end for Rust Chess, and it will help chess enthusiasts all over the world to analyze their games conveniently, fast, and thanks to the open-source community it will be alive and expanding for years to come.

List of tables

Table 1. python-chess [interpreter] average performance.....	40
Table 2. python-chess [PyPy] average performance	41
Table 3. Rust Chess average performance	42
Table 4. Rust Chess 489 MB file average performance per no. threads	43

List of figures

Figure 1. Chess analysis software (https://lichess.org/analysis)	7
Figure 2. Portable Game Notation, an exemplary game	8
Figure 3. En-passant capture. A move possible only right after opponent's double pawn move. (https://lichess.org/analysis).....	13
Figure 4. Draw by insufficient material. Neither side has the resources to deliver checkmate. (https://lichess.org/analysis).....	14
Figure 5. Knight on c3 cannot move due to absolute pin. (https://lichess.org/analysis).....	14
Figure 6. Chess bitboard.....	17
Figure 7. A bitboard storing occupied squares in the starting position.....	18
Figure 8. A bitboard storing position of knights in the starting position	18
Figure 9. A bitboard storing squares available for the undisrupted rook on the A1 square.....	18
Figure 10. A bitboard storing squares occupied by white pieces in the starting position.....	18
Figure 11. Bitboard Editor (https://bitboard-editor.vercel.app/)	22
Figure 12. Example Python class to transpile	24
Figure 13. Resulting Rust code	24
Figure 14. More complex Python example	24
Figure 15. Resulting Rust code	24
Figure 16. Windows Performance Analyzer GUI.....	25
Figure 17. Windows Performance Recorder GUI.....	25
Figure 18. A Python generator (https://github.com/niklasf/python-chess/blob/master/chess/__init__.py).....	26
Figure 19. A Rust generator using gen_iter	26
Figure 20. A struct Board containing a BaseBoard field	27
Figure 21. Exemplary edge case chess position.....	28

Figure 22. Data structures representing a PGN game	29
Figure 23. RAV in a PGN format	29
Figure 24. A representation of a visitor, parsing full PGN information	30
Figure 25. Set of visitor capabilities.....	30
Figure 26. Exemplary parsing speed using debug build	30
Figure 27. Exemplary parsing speed using a release build	31
Figure 28. Parse_square function of Rust Chess.....	33
Figure 29. QueryResult structure	33
Figure 30. make_api_call function.....	34
Figure 31. Example of a winning and drawn position (https://lichess.org/analysis)	35
Figure 32. Rust Chess communication with an engine	36
Figure 33. Scholar's mate in Rust Chess	37
Figure 34. python-chess [interpreter] 2.6 MB file parsing time.....	40
Figure 35. python-chess [PyPy] 2.6 MB file parsing time.....	41
Figure 36. Rust Chess 2.6 MB file parsing time	42
Figure 37. Median time of parsing comparison. Note the logarithmic scale on the vertical axis	45
Figure 38. Comparison of median times in a test simulating practical use.....	46
Figure 39. Rust Chess compared to Cute Chess.....	47

Bibliography

- [1] E. W. Weisstein, "'Perfect Information.' From MathWorld--A Wolfram Web Resource," [Online]. Available: <https://mathworld.wolfram.com/PerfectInformation.html>. [Accessed 4 1 2022].
- [2] C. Shannon, "Programming a Computer for Playing Chess," *Philosophical Magazine*, vol. 41, no. 314, 1950.
- [3] V. Allis, *Searching for Solutions in Games and Artificial Intelligence*, [PhD thesis, University of Limburg, Maastricht, Netherlands], 1994.
- [4] "Lomonosov Tablebases," [Online]. Available: <https://tb7.chessok.com/probe/3/61>. [Accessed 5 1 2022].
- [5] A. V. Aho, "Compilers: Principles, Techniques, and Tools," Pearson plc, 2006.
- [6] D. Chisnall, "C is not a Low-Level Language," *ACM Queue*, vol. 16, no. 2, 2018.
- [7] [Online]. Available: <https://hackernoon.com/why-is-python-so-slow-e5074b6fe55b>. [Accessed 24 1 2022].
- [8] D. M. Beazley, *Python Essential Reference*, 4th ed., Addison-Wesley Professional, 2009.
- [9] M. Hertz and E. D. Berger, "Quantifying the Performance of Garbage Collection vs. Explicit Memory Management," in *Proceedings of the 20th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications*, San Diego, 2005.
- [10] "IBM," [Online]. Available: <https://www.ibm.com/docs/en/zos-basic-skills?topic=zos-compiled-versus-interpreted-languages>. [Accessed 5 1 2022].
- [11] M. Jarvis, "Dicebreaker," [Online]. Available: <https://www.dicebreaker.com/games/chess/how-to/how-to-play-chess>. [Accessed 6 1 2022].
- [12] "Board Representation," [Online]. Available: https://www.chessprogramming.org/Board_Representation. [Accessed 6 1 2022].
- [13] "Board Representation," [Online]. Available: [https://en.wikipedia.org/wiki/Board_representation_\(computer_chess\)](https://en.wikipedia.org/wiki/Board_representation_(computer_chess)). [Accessed 6 1 2022].
- [14] R. Hyatt, "Rotated Bitmaps, A New Twist on an Old Idea," *ICAA Journal*, vol. 22, no. 4,

- 1999.
- [15] T. I. Fenner and M. Levene, "Move Generation with Perfect Hash Functions," *ICGA Journal*, vol. 31, no. 1, 2008.
 - [16] "Bitboards," [Online]. Available: <https://www.chessprogramming.org/Bitboards>. [Accessed 8 1 2022].
 - [17] J. Pettersson, "[Guide] Move generation," [Online]. Available: <http://mediocrechess.blogspot.com/2006/12/guide-move-generation.html>. [Accessed 1 8 2022].
 - [18] B. F. Andrés and M. Pérez, "Transpiler-based architecture for multi-platform web applications," in *2017 IEEE Second Ecuador Technical Chapters Meeting (ETCM)*, 2017.
 - [19] E. Gamma, R. Helm, R. Johnson and J. M. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.
 - [20] "The Cargo Book," [Online]. Available: <https://doc.rust-lang.org/cargo/reference/profiles.html>. [Accessed 17 1 2022].
 - [21] "The Rust Performance Book," [Online]. Available: <https://nnethercote.github.io/perf-book.html>. [Accessed 17 1 2022].
 - [22] S. Al-Kuwari, J. H. Davenport and R. J. Bradford, "Cryptographic Hash Functions: Recent Design Trends and Security Notions," *Cryptology ePrint Archive*, 2011.
 - [23] "Game solved (After 122 years)," [Online]. Available: <https://www.chess.com/forum/view/general/game-solved-after-122-years>. [Accessed 28 1 2022].
 - [24] "PGN Mentor," [Online]. Available: <https://www.pgnmentor.com/files.html>. [Accessed 18 1 2022].
 - [25] [Online]. Available: <http://smallchess.com/Games/>. [Accessed 18 1 2022].
 - [26] "Amdahl's law," [Online]. Available: https://en.wikipedia.org/wiki/Amdahl%27s_law. [Accessed 19 1 2022].
 - [27] "PyPy's documentation," [Online]. Available: <https://doc.pypy.org/en/latest/faq.html#does-pypy-have-a-gil-why>. [Accessed 22 1 2022].
 - [28] "Building Jerry," [Online]. Available: <https://buildingjerry.wordpress.com/2020/02/29/towards-fast-pgn-parsing/>. [Accessed 13

1 2022].

[29] "Web Archive - Internet Archive," [Online]. Available:
<https://web.archive.org/web/20190213033051/https://rebel13.nl/download/data.html>.

[Accessed 23 1 2022].

[30] "multiprocessing — Process-based parallelism," [Online]. Available:
<https://docs.python.org/3/library/multiprocessing.html>. [Accessed 26 1 2022].